

O'ZBEKISTON RESPUBLIKASI OLIY TA'LIM, FAN VA INNOVATSIYALAR
VAZIRLIGI
MUHAMMAD AL-XORAZMIY NOMIDAGI TOSHKENT AXBOROT
TEXNOLOGIYALARI UNIVERSITETI
SAMARQAND FILIALI



KOMPYUTER INJINIRINGI FAKULTETI

INDIVIDUAL LOYIHA ISHI

Bajardi: K1023-01 Xudoyqulov Saidabbos.

Qabul qildi: _____.

Loyiha to'liq kodlari repozitoriyasi:

<https://github.com/SaidAbbos96/obyektlarni-tanish-tizimi>

SAMARQAND – 2026

TASVIRLARDAN OBYEKTЛАRNI TANIB OLISH

DASTURINI YARATISH

Mazkur individual loyiha ishining maqsadi tasvirlardan obyektlarni tanib olish jarayonini avtomatlashtiruvchi dasturiy tizim yaratishdan iborat. Loyihada kompyuter ko‘rish texnologiyalari, tasvirlarni qayta ishlash usullari va sun‘iy intellekt algoritmlarining amaliy qo‘llanilishi o‘rganilgan. Ishlab chiqilgan dastur turli toifadagi obyektlarni aniqlash, natijalarni vizual ko‘rsatish hamda ularni saqlash imkoniyatiga ega. Olingan natijalar tizimning amaliy masalalarni yechishda samarali qo‘llanishi mumkinligini ko‘rsatadi.

MUNDARIJA

Оглавление

I BOB. KIRISH	5
1.1. Dolzarblik.....	5
1.2. Maqsad	5
1.3. Vazifalar	5
1.4. Tadqiqot obyekti.....	6
1.5. Tadqiqot predmeti	6
1.6. Amaliy ahamiyati	6
1.7. Ishning tuzilishi	7
1.8. Tadqiqot metodologiyasi	7
1.9. Muammoning amaliy qoyilishi	8
1.10. Kutilayotgan ilmiy-amaliy natijalar	8

1.11. Ishning chegaralari	8
2-BOB. TASVIRLARDAN OBYEKTЛАRNI TANIB OLISHNING	
NAZARIY ASOSLARI	10
2.1. Kompyuter ko'rish tushunchasi	10
2.2. Sun'iy intellekt va mashinali o'rganish	10
2.3. Tasvirlarni qayta ishlash texnologiyalari.....	11
2.4. Obyektlarni aniqlash algoritmlari	12
2.5. YOLO algoritmlari oilasi.....	12
2.6. YOLOv8 modeli.....	13
2.7. OpenCV kutubxonasi	13
2.8. Mavjud tizimlar tahlili	14
2.9. Klassik va chuqur organish usullarini taqqoslash.....	14
2.10. Amaliy loyiha uchun tanlov sababi.....	15
2.11. Obyekt aniqlashda baholash korsatkichlari	16
2.12. Nazariy bob bo'yicha yakuniy mulohaza.....	17
2.13. Qisqa xulosa	17
III-BOB. DASTURIY TIZIMNI LOYIHALASH.....	19
3.1. Texnik topshiriq	19
3.2. Asosiy funksional nuqtalar	19
3.3. Cheklovlar	19
3.4. Funksional talablar	19
3.5. Nofunksional talablar.....	20
3.6. Tizim arxitekturas.....	20
3.7. Loyihaning katalog tuzilmasi.....	21

3.8. Ma'lumotlar oqimi	22
3.9. UML diagrammalar	23
3.10. Texnologiyalarni tanlash sabablari.....	23
3.11. Talablar matritsasi	24
3.12. Komponentlararo bog'lanishning amaliy modeli	24
3.13. Xavflar va ularni boshqarish rejasi.....	25
3.14. Loyihalash bobi bo'yicha qo'shimcha yakun	26
IV-BOB. DASTURIY TA'MINOTNI ISHLAB CHIQISH	27
4.1. Loyiha strukturasi	27
4.2. Asosiy modul.....	27
4.3. Interfeys modullari	28
4.4. Rasm yuklash moduli	30
4.5. Obyekt aniqlash moduli	30
4.6. Natijalarni saqlash moduli	32
4.7. Jurnal yuritish moduli	33
4.8. Test modullari.....	33
4.9. Hodisa asosidagi boshqaruv mexanizmi	34
4.10. Asinxron aniqlash oqimi	35
4.11. Xatolarni boshqarish strategiyasi	36
4.12. Kod sifati va uslubiy tamoyillar	36
4.13. Ishlash samaradorligini oshirish bo'yicha choralar	37
4.14. 4-bob bo'yicha kengaytirilgan yakun	37
4.15. 4-bob bo'yicha xulosa	37

V-BOB. SINOV NATIJALARI VA TAHLIL	38
5.1. Sinov muhiti	38
5.2. Test ma'lumotlari	38
5.3. Test natijalari.....	39
5.4. Aniqlik ko'rsatkichlari	50
5.5. Baholash metodikasi.....	51
5.6. Haar va DNN strategiyalarini amaliy taqqoslash	51
5.7. Afzalliklar	52
5.8. Kamchiliklar.....	53
5.9. Kelajakdagi rivojlantirish	53
5.10. VI-bob bo'yicha kengaytirilgan tahliliy yakun	54
5.11. V-bob bo'yicha xulosa	54
NATIJALAR	55
6.2. NATIJALARNING AMALIY QIYMATI.....	55
6.3. TALABLAR BAJARILISHI BO'YICHA QISQA HISOBOT	56
XULOSA.....	57
SHAXSIY O'QUV NATIJALARI	58
YAKUNIY XULOSA.....	58
FOYDALANILGAN ADABIYOTLAR.....	59
ILOVALAR.....	61

I BOB. KIRISH

1.1. Dolzarblik

Bugungi kunda tasvirlar bilan ishlash deyarli hamma sohada uchraydi: xavfsizlik tizimlari, transport monitoringi, ishlab chiqarishdagi nazorat, tibbiyot, hatto oddiy mobil ilovalarda ham. Tasvirdan kerakli obyektни tez va aniq ajratib olishning amaliy qiymati juda yuqori. Odatda bu ish inson tomonidan qilinsa, vaqt ko'p ketadi va subyektiv xatolar bo'lishi mumkin. Shu sababli obyektlarni avtomatik tanib olish dasturlari dolzarb yo'nalish hisoblanadi.

Ushbu individual loyihada aynan shu masala tanlandi: rasm yuklanadi, dastur esa tanlangan algoritm asosida obyektlarni topadi, koordinatalarini belgilaydi, natijani jadval ko'rinishida ko'rsatadi va saqlab beradi. Men loyihani oddiy foydalanuvchi ham ishlata oladigan darajada qulay qilishga harakat qildim.

1.2. Maqsad

Loyihaning asosiy maqsadi — Python va OpenCV asosida tasvirlardan obyektlarni tanib oladigan, grafik interfeysga ega bo'lgan, amaliy ishlashga tayyor dastur yaratish.

1.3. Vazifalar

Maqsadga erishish uchun quyidagi vazifalar belgilandi:

1. Obyekt aniqlash bo'yicha nazariy manbalarni o'rganish.
2. Haar Cascade va DNN (MobileNet-SSD) yondashuvlarini tahlil qilish.
3. Dastur uchun texnik topshiriq va talablarni shakllantirish.

4. Modulli arxitektura asosida tizim dizaynini ishlab chiqish.
5. GUI, rasm yuklash, aniqlash va saqlash modullarini yozish.
6. Test rasmlar asosida sinov o'tkazish va natijalarni tahlil qilish.
7. Yakuniy texnik hisobotni rasmiylashtirish.

1.4. Tadqiqot obyekti

Tadqiqot obyekti sifatida raqamli tasvirlar va ulardagi real obyektlar (transport, hayvonlar, inson, mebel va boshqalar) olindi.

1.5. Tadqiqot predmeti

Tadqiqot predmeti — tasvirlarda obyektlarni aniqlash algoritmlarini dasturiy realizatsiya qilish va ularning amaliy ishlash samaradorligini baholash.

1.6. Amaliy ahamiyati

Mazkur loyiha quyidagi amaliy natijalarni beradi:

- Kompyuter ko'rish bo'yicha o'quv-amaliy ko'nikmalar shakllanadi.
- Obyekt aniqlashning klassik va chuqur o'rganishga asoslangan usullarini bir dasturda solishtirish imkoni paydo bo'ladi.
- Dastur laboratoriya ishlari, kurs loyihalari va kichik tadqiqot ishlarida tayyor platforma sifatida ishlatilishi mumkin.
- Natijalarni JSON formatda saqlash hisobiga keyingi statistik tahlil yoki boshqa tizimga integratsiya qilish osonlashadi.

1.7. Ishning tuzilishi

Hisobot kirish, to'rtta asosiy bob, natijalar, xulosa, foydalanilgan adabiyotlar va ilovalardan iborat. 1-bobda nazariy asoslar yoritiladi. 2-bobda tizimni loyihalash qarorlari beriladi. 3-bobda to'g'ridan-to'g'ri kod darajasidagi realizatsiya tushuntiriladi. 4-bobda sinov natijalari va tahlil keltiriladi. Yakunda umumiy xulosa, manbalar va qo'shimchalar beriladi.

1.8. Tadqiqot metodologiyasi

Ushbu ishda nazariy tahlil va amaliy implementatsiya birga olib borildi. Dastlab kompyuter korish va obyekt aniqlash bo'yicha asosiy ilmiy manbalar o'rganildi. Keyingi bosqichda esa real kod yozish orqali nazariy qarashlar tekshirildi. Men ishni quyidagi ketma-ketlikda tashkil qildim:

1. Muammoni aniq ifodalash.
2. Mavjud yechimlarni tahlil qilish.
3. Loyihaga mos texnologiyalarni tanlash.
4. Modulli arxitektura yaratish.
5. Kod yozish va testlar bilan tekshirish.
6. Turli rasmlar orqali tajriba o'tkazish.
7. Olingan natijalarni tahlil qilib, xulosa chiqarish.

Bu yondashuvning afzalligi shundaki, har bir bosqich oldingi bosqichdagi qarorlarni tekshiradi. Masalan, nazariyada tanlangan algoritm amaliy kodda kutilgan natijani bermasa, qayta baholash imkoniyati paydo bo'ladi.

1.9. Muammoning amaliy qoyilishi

Loyihada yechilayotgan muammo oddiy ko'rinishda bo'lsa ham, ichki tomondan bir nechta murakkabliklarni o'z ichiga oladi. Rasmda obyektни topish uchun nafaqat model kerak, balki foydalanuvchi bilan to'g'ri muloqot qiladigan interfeys, xatolarni tushunarli ko'rsatadigan mexanizm va natijalarni saqlash qismi ham bo'lishi shart.

Shuning uchun dastur quyidagi amaliy ssenariyga moslab qurildi: foydalanuvchi rasm tanlaydi, strategiya belgilaydi, aniqlashni ishga tushiradi, natijani koradi va saqlaydi. Bu jarayon davomida tizim xato bo'lsa ham to'xtab qolmasdan, foydalanuvchiga nima sabab bo'lganini bildirib turadi.

1.10. Kutilayotgan ilmiy-amaliy natijalar

Loyihadan quyidagi natijalar kutilgan va amalda tasdiqlangan:

- obyekt aniqlashning ikki yondashuvi bitta tizimda ishlashi;
- dastur modullari orasida aniq mas'uliyat taqsimoti bo'lishi;
- interfeys orqali texnik bo'lmagan foydalanuvchi ham ishlata olishi;
- test rasmlarda barqaror natija qaytarishi;
- saqlangan JSON natijalari asosida keyingi tahlil o'tkazish imkoniyati.

Bu natijalar loyiha nafaqat bir martalik demo ekanini, balki keyinchalik kengaytiriladigan platforma bo'la olishini ko'rsatadi.

1.11. Ishning chegaralari

Har qanday amaliy ishda bo'lgani kabi bu loyihada ham aniq chegaralar bor.

Ular hisobotning keyingi bo'limlarida to'g'ri baholash uchun muhim:

1. Dastur hozircha statik rasmlar bilan ishlaydi.
2. DNN strategiyasi uchun model fayllari alohida yuklanadi.
3. Anqlik qiymati rasm sifati, yorug'lik va fon murakkabligiga bog'liq.
4. Video oqim va real-time tracking qismi hozircha qoshilmagan.
5. YOLOv8 strategiyasi arxitektura darajasida tayyorlangan, ammo to'liq integratsiya keyingi bosqichga qoldirilgan.

Shu bilan birga, aynan ushbu chegaralar kelajakdagi rivojlantirish uchun aniq yonalishlarni ham beradi.

II-BOB. TASVIRLARDAN OBYEKTЛАRNI TANIB OLISHNING NAZARIY ASOSLARI

2.1. Kompyuter ko'rish tushunchasi

Kompyuter ko'rish (Computer Vision) — bu kompyuter tizimlarining tasvir va videodan ma'no ajratib olishiga qaratilgan yo'nalish hisoblanadi. Inson ko'rish tizimi qanday qilib tashqi muhitni kuzatib, predmetlarni farqlasa, kompyuter ko'rish ham shunga o'xshash vazifani matematik modellar va algoritmlar orqali bajaradi.

Kompyuter ko'rishda odatda quyidagi asosiy bosqichlar bo'ladi:

1. Tasvirni olish.
2. Oldindan qayta ishlash.
3. Muhim belgilarni ajratish.
4. Tasniflash yoki aniqlash.
5. Natijani vizuallashtirish.

Loyihamda aynan shu zanjirning to'liq amaliy ko'rinishi bor: rasm yuklanadi, o'lchamga moslashtiriladi, model orqali obyektlar aniqlanadi, natija ramkalar bilan foydalanuvchiga ko'rsatiladi.

2.2. Sun'iy intellekt va mashinali o'rganish

Sun'iy intellekt (AI) keng tushuncha bo'lib, mashinali o'rganish (Machine

Learning) uning asosiy yo'nalishlaridan biridir. Tasvirdan obyekt aniqlash masalasida mashinali o'rganish, ayniqsa chuqur o'rganish (Deep Learning) juda katta samara berdi.

Klassik usullarda (masalan, Haar Cascade) oldindan belgilangan xususiyatlar asosida aniqlash bajariladi. Chuqur o'rganishda esa model katta hajmdagi ma'lumotlardan xususiyatlarni o'zi o'rganadi. Natijada model turli holatlardagi obyektlarni yaxshiroq umumlashtira oladi.

Ushbu loyihada ikkala yondashuvni ham bir tizim ichida berishimning sababi — taqqoslash imkonini ochish va foydalanuvchiga vazifaga mos usulni tanlash erkinligini berishdir.

2.3. Tasvirlarni qayta ishlash texnologiyalari

Tasvirni qayta ishlash obyekt aniqlashning ajralmas qismi hisoblanadi. Rasm sifati, o'lchami, shovqin darajasi model natijasiga bevosita ta'sir qiladi. Shu sababli real tizimlarda oldindan qayta ishlash bosqichlari bo'ladi:

- o'lchamni normalizatsiya qilish;
- rang fazosini o'zgartirish;
- filtratsiya;
- kontrastni oshirish;
- segmentatsiya.

Mening amaliy dasturimda eng muhim texnik qadam — turli o'lchamdagi rasmlarni tizim yuklay oladigan chegaraga keltirish. Bu xotira sarfini kamaytiradi

va aniqlash jarayonini tezlashtiradi.

2.4. Obyektlarni aniqlash algoritmlari

Obyekt aniqlash algoritmlari odatda ikki guruhga bo'linadi:

1. Klassik algoritmlar (Haar Cascade, HOG+SVM va boshqalar).
2. Chuqur o'rganish algoritmlari (R-CNN oilasi, SSD, YOLO oilasi).

Klassik usullar yengil va tez, ammo murakkab sahnalarda aniqligi pasayadi. Chuqur o'rganish usullari esa ko'proq hisoblash resursi talab qilsa ham, sinflar soni ko'p bo'lgan vazifalarda yuqori natija beradi.

Loyihadagi Haar strategiyasi oddiy yuz aniqlash uchun, DNN strategiyasi esa ko'p sinfli aniqlash uchun ishlatiladi.

2.5. YOLO algoritmlari oilasi

YOLO (You Only Look Once) algoritmi obyekt aniqlashda eng mashhur yondashuvlardan biri. Uning asosiy afzalligi — bir marta o'tishda obyektning joylashuvi va sinfini birga topishi. Bu real vaqt tizimlarida katta ustunlik beradi.

YOLO oilasi versiyalari rivojlanishi davomida quyidagi yo'nalishlar kuchaydi:

- aniqlik va tezlik muvozanati;
- kichik obyektlarni yaxshi topish;
- treningni soddalashtirish;

- deployment jarayonini yengillashtirish.

Garchi joriy loyiha asosan Haar va MobileNet-SSD bilan qurilgan bo'lsa ham, arxitektura keyinchalik YOLO modelini qo'shishga mos holda tashkil etilgan.

2.6. YOLOv8 modeli

YOLOv8 — YOLO oilasining zamonaviy versiyalaridan biri bo'lib, segmentatsiya, klassifikatsiya va aniqlash vazifalarini qo'llab-quvvatlaydi. Ushbu modelning amaliy afzalliklari:

- yuqori aniqlik;
- oson API;
- turli hajmdagi model variantlari (nano, small, medium va h.k.);
- edge qurilmalarga moslashuvchanlik.

Kelajakdagi rivojlantirish bosqichida loyihaga YOLOv8 ni strategiya sifatida qo'shish rejalashtiriladi. Bunda mavjud abstrakt interfeys saqlanadi, yangi aniqlovchi klass qo'shiladi.

2.7. OpenCV kutubxonasi

OpenCV (Open Source Computer Vision Library) tasvirlarni qayta ishlash va kompyuter ko'rish bo'yicha eng ko'p qo'llanadigan kutubxonalardan biri. Kutubxona C++ da yozilgan bo'lsa-da, Python interfeysi juda qulay.

Loyihada OpenCV quyidagi vazifalarda ishlatildi:

- rasm yuklash (“cv2.imread”);
- rasmga ramka va matn chizish (“cv2.rectangle”, “cv2.putText”);
- Haar model yuklash (“cv2.CascadeClassifier”);
- DNN tarmoqni ishga tushirish (“cv2.dnn.readNetFromCaffe”);
- natija rasmini saqlash (“cv2.imwrite”).

Bu kutubxona platformalararo ishlashi va keng hujjat bazasi bilan amaliy loyihalar uchun juda qulay ekanligi amalda tasdiqlandi.

2.8. Mavjud tizimlar tahlili

Mavjud obyekt aniqlash tizimlarini tahlil qilganda, ular odatda ikki xil bo'ladi: yirik bulut xizmatlari va yengil lokal dasturlar. Bulut xizmatlari imkoniyati keng, lekin internet va pullik tarifga bog'liq. Lokal dasturlar esa mustaqil ishlaydi, ammo odatda funksiyasi cheklangan bo'ladi.

Mening loyiham lokal ishlashga yo'naltirilgan. Bu quyidagi afzalliklarni beradi:

- internetga qaramlik yo'q;
- ma'lumotlar lokal qoladi;
- ta'lim jarayonida oson ko'rsatma sifatida ishlatish mumkin.

Shu bilan birga, loyiha arxitekturasi modul ko'rinishda qurilgani sababli kelajakda bulut API bilan integratsiya qilish ham mumkin.

2.9. Klassik va chuqur organish usullarini taqqoslash

Ushbu bo'limda obyekt aniqlashda eng ko'p uchraydigan ikki yondashuv amaliy nuqtai nazardan taqqoslanadi: klassik usullar va chuqur o'rganish usullari. Nazariyada ikkalasining ham o'rni bor, lekin qaysi birini tanlash vazifaga bog'liq.

Taqqoslash mezonlari

Mezoni	Klassik usullar (Haar, HOG)	Chuqur o'rganish (SSD, YOLO)
Model hajmi	Kichik	Odatda katta
Ishga tushirish soddaligi	Oson	Nisbatan murakkab
Ko'p sinfli aniqlash	Cheklangan	Kuchli
Murakkab fonlarda barqarorlik	Past/o'rta	O'rta/yuqori
Kichik obyektlarni topish	Qiyin	Modelga bog'liq holda yaxshi
O'quv ma'lumotiga ehtiyoj	Kamroq	Ko'proq
Tez prototiplash	Tez	O'rta

Ko'rinib turibdiki, klassik usullar oddiy vazifalarda juda qulay. Masalan, yuz aniqlash uchun Haar ba'zi holatlarda yetarli bo'ladi va qurilma resursini kam talab qiladi. Ammo sinflar soni oshganda, chuqur o'rganish usullari ustunlik qiladi.

2.10. Amaliy loyiha uchun tanlov sababi

Loyihada ikki usul birga berilishining asosiy sabablari:

1. Ta'limiy nuqtai nazardan farqni real ko'rsatish.
2. Har xil qurilma sharoitida mos variantni tanlash.

3. Kelajakda modern model qo'shish uchun asos tayyorlash.

Bu yechim hisobiga foydalanuvchi oldindan qattiq bitta algoritmgga bog'lanib qolmaydi.

2.11. Obyekt aniqlashda baholash kursoratkichlari

Obyekt aniqlash tizimini baholashda faqat "topdi" yoki "topmadi" deyish yetarli emas. Amalda bir nechta metrika bilan ishlanadi. Quyida eng muhim kursoratkichlar oddiy til bilan izohlanadi.

Precision va Recall

- Precision: topilgan obyektlarning nechta qismi to'g'ri ekanini bildiradi.
- Recall: asl obyektlarning nechta qismi topilganini bildiradi.

Agar model juda ehtiyotkor bo'lsa, precision yuqori, recall past bo'lishi mumkin. Aksincha, hamma joydan obyekt qidirsang, recall yuqori bo'lib, precision pasayishi mumkin.

IoU (Intersection over Union)

IoU ramkaning qanchalik to'g'ri tushganini ko'rsatadi. Model chizgan ramka va haqiqiy ramka kesishmasi qancha katta bo'lsa, IoU yuqori bo'ladi. Odatda $\text{IoU} \geq 0.5$ bo'lsa aniqlash qoniqarli deb olinadi.

Precision va recall o'rtasidagi balansni ko'rsatish uchun F1-score ishlatiladi:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Bu metrika ayniqsa sinflar notekis bo'lgan holatda foydali.

mAP (mean Average Precision)

Ko'p sinfli vazifalarda eng ko'p ishlatiladigan umumiy metrika. Har bir sinf bo'yicha AP hisoblanib, keyin o'rtachasi olinadi. mAP qanchalik yuqori bo'lsa, model shunchalik yaxshi ishlaydi.

Mazkur individual loyihada to'liq mAP hisoblash pipeline'i qurilmagan bo'lsa ham, sinf bo'yicha amaliy natijalar jadvallari berildi. Bu keyingi bosqichda to'liq metrik baholashga o'tish uchun tayyor asos hisoblanadi.

2.12. Nazariy bob bo'yicha yakuniy mulohaza

Nazariy tahlilning asosiy natijasi shuki, obyekt aniqlashda "eng yaxshi yagona usul" yoq. Vazifa, qurilma imkoniyati, kerakli tezlik va aniqlik darajasiga qarab yondashuv tanlanadi. Shu nuqtai nazardan loyiha arxitekturasini to'g'ri tanlangan: u bir tomondan yengil va tez ishlaydigan klassik usulni, ikkinchi tomondan ko'p sinfli chuqur modelni qo'llash imkonini beradi.

2.13. Qisqa xulosa

Nazariy tahlil shuni ko'rsatdiki, obyekt aniqlash uchun bitta universal yechim yo'q. Vazifaga qarab klassik yoki chuqur o'rganish usulini tanlash kerak. Ushbu loyiha shu tamoyilga asoslanib, ikki strategiyani bitta tizimda birlashtirdi. Keyingi bobda ushbu yechim qanday loyihalanganligi bosqichma-bosqich

bayon qilinadi.

III-BOB. DASTURIY TIZIMNI LOYIHALASH

3.1. Texnik topshiriq

Ushbu bo'limda dasturdan kutilgan asosiy funksiyalar va cheklovlar belgilandi. Texnik topshiriqning asosiy g'oyasi shundan iborat bo'ldiki, foydalanuvchi rasmni yuklay olishi, strategiyani tanlashi, obyektlarni aniqlashi va natijani saqlashi kerak. Interfeys sodda, tushunarli va barqaror bo'lishi talab qilindi.

3.2. Asosiy funksional nuqtalar

- Rasm faylini tanlash yoki drag&drop orqali yuklash.
- Aniqlash strategiyasini almashtirish (“haar” yoki “dnn”).
- Aniqlashni asinxron bajarish.
- Aniqlangan obyektlarni rasm ustida ramkalar bilan ko'rsatish.
- Natijalarni jadval va statistika ko'rinishida chiqarish.
- Natijani PNG va JSON formatida saqlash.

3.3. Cheklovlar

- Dastur desktop rejimida ishlaydi.
- DNN modeli uchun tashqi “.caffemodel” fayl talab qilinadi.
- Juda katta rasmlar avtomatik o'lchamga keltiriladi.

3.4. Funksional talablar

Funksional talablar quyidagi tarzda guruhlandi:

1. Yuklash funksiyasi: qo'llab-quvvatlanadigan formatlarni tekshirish.
2. Aniqlash funksiyasi: tanlangan strategiya asosida obyektlarni topish.
3. Vizualizatsiya funksiyasi: ramka, sinf nomi va aniqlik darajasini chizish.
4. Saqlash funksiyasi: natijani ikki formatda eksport qilish.
5. Boshqaruv funksiyasi: strategiya almashtirish va holatni ko'rsatish.

3.5. Nofunksional talablar

- Tezkorlik: odatiy rasmlarda aniqlash jarayoni bir necha soniyadan oshmasligi.
- Foydalanish qulayligi: GUI orqali ko'p bosqichsiz ishlash.
- Kengaytiriluvchanlik: yangi model qo'shish imkoniyati bo'lishi.
- Xatoga chidamlilik: noto'g'ri format, topilmagan fayl va model xatolari aniq xabar bilan qaytarilishi.
- Kod tartibi: modulga bo'lingan, o'qilishi oson arxitektura.

3.6. Tizim arxitekturasi

Loyihada bir nechta arxitektura g'oyalari birga qo'llandi:

- Clean Architecture — modullar mas'uliyatini aniq ajratish uchun.
- Strategy Pattern — aniqlash algoritmlarini almashtirish uchun.
- MVP (Model-View-Presenter) — GUI va biznes logikani bo'lish uchun.

flowchart LR

A[Interfeys GUI] --> B[Taqdimotchi]

B --> C[Rasm Yuklash Xizmati]

B --> D[Obyekt Aniqlash Xizmati]

B --> E[Natija Saqlash Xizmati]

D --> F[Haar Strategiyasi]

D --> G[DNN Strategiyasi]

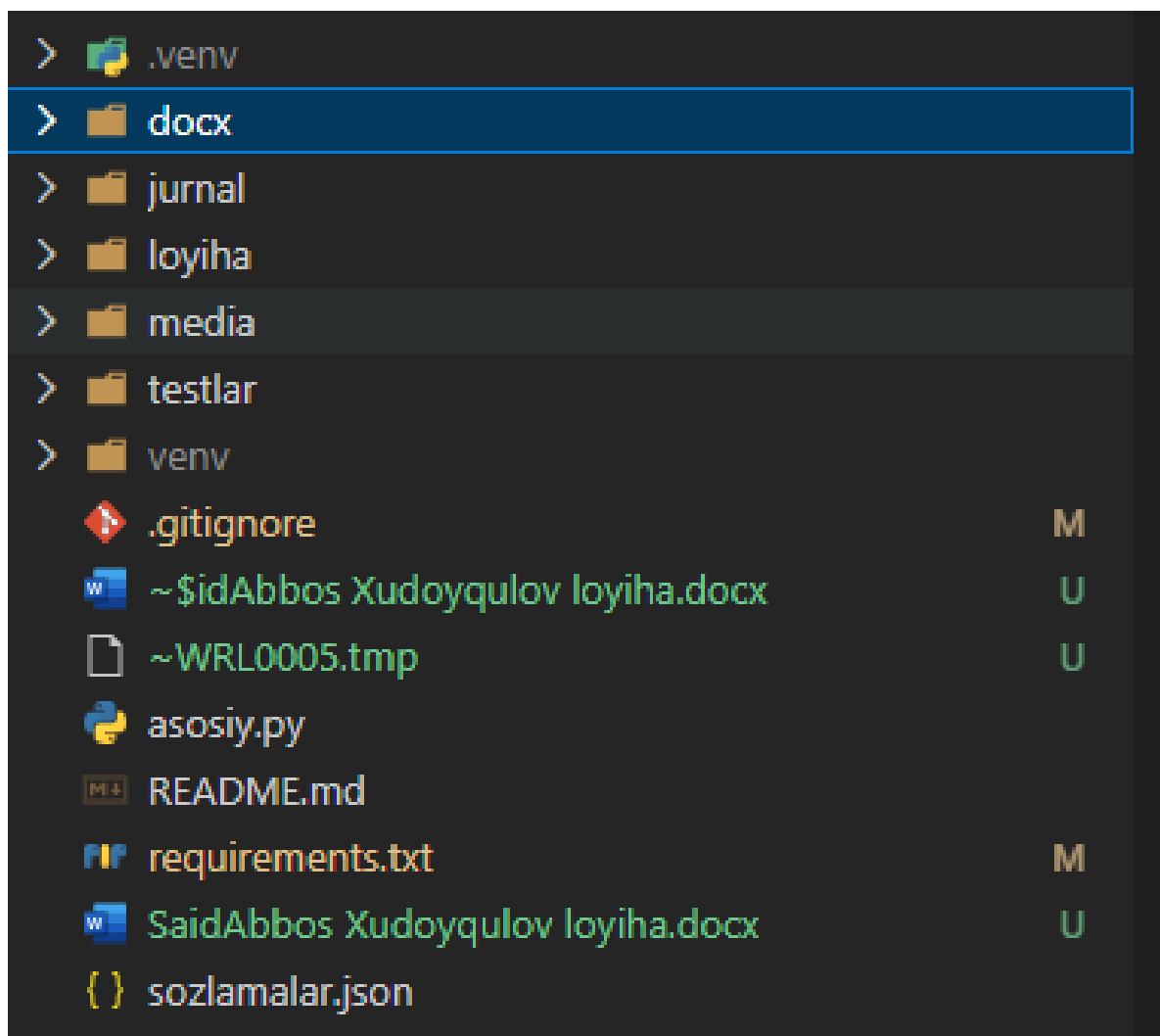
D --> H[Aniqlash Natijasi modeli]

Bu arxitektura loyihani keyin kengaytirishni ancha osonlashtirdi.

3.7. Loyihaning katalog tuzilmasi

Katalog strukturasi amaliy mas'uliyat bo'yicha ajratilgan:

- “interfeys/” — foydalanuvchi bilan muloqot qismi.
- “taqdimotchi/” — GUI va xizmatlar o'rtasidagi bog'lovchi qatlam.
- “xizmatlar/” — asosiy biznes logika.
- “aniqlash/” — strategiyalar (Haar, DNN).
- “modellar/” — domen obyektlari.
- “sozlamalar/” — konfiguratsiya va default parametrlar.
- “vositalar/” — yordamchi utilitalar.



3.8. Ma'lumotlar oqimi

Rasm yuklashdan natija saqlashgacha oqim quyidagicha:

1. GUI rasm manzilini taqdimotchiga uzatadi.
2. Taqdimotchi rasm yuklash xizmatini chaqiradi.
3. Aniqlash tugmasi bosilganda taqdimotchi aniqlash xizmatini asinxron ishga tushiradi.
4. Xizmat strategiya orqali obyektlarni topadi va “AniqlashNatijasi” yaratadi.
5. GUI natijani jadval va rasm ko'rinishida aks ettiradi.
6. Saqlash so'rovida PNG va JSON fayl yoziladi.

3.9. UML diagrammalar

Use Case (soddalashtirilgan)

flowchart TD

U[Foydalanuvchi] --> A[Rasm yuklash]

U --> B[Strategiya tanlash]

U --> C[Aniqlashni boshlash]

U --> D[Natijani saqlash]

Class aloqalari (soddalashtirilgan)

classDiagram

class AsosiyOyna

class AsosiyTaqdimotchi

class ObyektAniqlovchiXizmat

class AniqlashStrategiyasi

class HaarAniqlovchi

class DnnAniqlovchi

AsosiyOyna --> AsosiyTaqdimotchi

AsosiyTaqdimotchi --> ObyektAniqlovchiXizmat

ObyektAniqlovchiXizmat --> AniqlashStrategiyasi

AniqlashStrategiyasi <|-- HaarAniqlovchi

AniqlashStrategiyasi <|-- DnnAniqlovchi

3.10. Texnologiyalarni tanlash sabablari

- Python: tez prototiplash, kuchli kutubxona ekotizimi.

- OpenCV: obyekt aniqlash va tasvir qayta ishlashda standart vosita.
- Tkinter: desktop GUI uchun yengil va oson.
- NumPy: tasvir ma'lumotlarini massiv sifatida samarali boshqarish.
- Pytest: unit testlarni qulay yuritish.

3.11. Talablar matritsasi

Loyihalash sifatini tekshirish uchun talablar matritsasi tuzildi. Bu usulda har bir talabga mos modul yoki funksiya biriktiriladi. Natijada qaysi talab kodda qayerda bajarilganini kuzatish oson bo'ladi.

TALAB KODI	TALAB MAZMUNI	AMALGA OSHIRILGAN JOY
FR-01	Rasm yuklash	"RasmYuklashXizmati.yuklash()"
FR-02	Strategiya almashtirish	"AsosiyTaqdimotchi.strategiyani_almashtirish()"
FR-03	Obyekt aniqlash	"ObyektAniqlovchiXizmat.aniquash()"
FR-04	Natijani jadvalda ko'rsatish	"NatijaPanel.natijani_korsatish()"
FR-05	Natijani saqlash	"NatijaSaqlashXizmati.saqlash()"
NFR-01	Kodning modulliligi	"loyiha/" papka arxitekturasi
NFR-02	Xatoni qayta ishlash	"xatolar.py" va hodisa tizimi
NFR-03	Interfeysning javob beruvchanligi	asinxron aniqlash + navbat mexanizmi

Talablar matritsasi keyingi himoya paytida ham juda foydali bo'ladi, chunki savol berilganda talabning amaliy dalilini tez ko'rsatish mumkin.

3.12. Komponentlararo bog'lanishning amaliy modeli

Arxitekturadagi eng muhim qarorlardan biri - modullar o'rtasidagi bog'lanishni minimal ushlab turish bo'ldi. Shu sababli komponentlar bir-biriga to'g'ridan-to'g'ri emas, asosan xizmat interfeyslari orqali murojaat qiladi.

Bog'lanish qoidalar

1. Interfeys qatlami biznes hisob-kitob qilmaydi.
2. Taqdimotchi qatlam GUI detallariga ortiqcha bog'lanmaydi.
3. Aniqlash xizmati strategiya bilan abstrakt shartnoma orqali ishlaydi.
4. Saqlash qatlami aniqlash algoritmidan mustaqil qoladi.

Bu qoidalar sababli kodni o'zgartirishda "domino effekti" kamayadi, ya'ni bitta modul o'zgarsa ham boshqa modullarni buzish xavfi past bo'ladi.

3.13. Xavflar va ularni boshqarish rejasi

Loyihalash bosqichida texnik xavflar oldindan yozib chiqildi:

Xavf	Ta'siri	Ehtimol	Qarshi chora
DNN model topilmasligi	DNN ishlamaydi	O'rta	READMEda aniq yo'riqnoma + ogohlantirish dialogi
Katta rasmda sekinlashish	UX yomonlashadi	O'rta	avtomatik o'lchamga keltirish
GUI muzlashi	foydalanuvchi ishlata olmaydi	Past/o'rta	asinxron aniqlash va navbat
Noto'g'ri formatli fayl	xatoliklar ko'payadi	Yuqori	format validatsiyasi
Kod murakkablashuvi	keyingi rivojlantirish qiyinlashadi	O'rta	qatlamli arxitektura

Bu jadval real ishlab chiqish jarayonida o'zini oqladi. Masalan, DNN model

yo'q holat uchun dastur qulab tushmasdan foydalanuvchiga ogohlantirish beradi.

3.14. Loyihalash bobi bo'yicha qo'shimcha yakun

Loyihalash bosqichida tanlangan yechimlar keyingi implementatsiyada barqaror ishladi. Ayniqsa Strategy Pattern va MVP birga qo'llangani tufayli algoritm almashinuvi ham, interfeysni rivojlantirish ham alohida yo'nalishda olib borildi. Bu esa texnik qarorlarning to'g'ri tanlanganini amaliy jihatdan isbotlaydi.

3-bob bo'yicha xulosa

Loyihalash bosqichida asosiy e'tibor kodni kengaytiriladigan va tushunarli qilishga qaratildi. Tanlangan arxitektura qarorlari keyinchalik YOLO kabi yangi model qo'shish uchun ham tayyor asos yaratdi.

IV-BOB. DASTURIY TA'MINOTNI ISHLAB CHIQISH

Bu bobda loyihadagi har bir asosiy modul alohida yoritiladi. Har bo'limda modulning qaysi faylda joylashgani, vazifasi, ishlash mexanizmi va muhim kod parchasi keltiriladi.

4.1. Loyiha strukturasi

Joylashuvi: loyiha ildiz katalogi va “loyiha/” papkasi

Vazifasi: tizimni qatlamlar bo'yicha tartibli tashkil qilish

Qanday ishlaydi: har bir papka alohida mas'uliyatga ega bo'lib, kod aralashib ketmaydi

Loyiha ichidagi strukturaga qarab, GUI kodi va biznes logika qat'iy ajratilgan. Bu xatolarni topishni yengillashtiradi.

4.2. Asosiy modul

Fayl: “asosiy.py”

Vazifasi: dasturni ishga tushirish, sozlamalarni yuklash, papkalarni tayyorlash, jurnalni yoqish

Ishlash tartibi: “asosiy()” funksiyasi ichida ketma-ket tayyorlov bosqichlari bajarilib, oxirida GUI ochiladi

```
from loyiha.sozlamalar.sozlamalar import Sozlamalar
from loyiha.xizmatlar.jurnal_xizmati import jurnal_sozlash
from loyiha.vositalar.fayl_vositlari import kerakli_papkalarni_yaratish
```

```

from loyiha.interfeys.asosiy_oyna import AsosiyOyna
def asosiy() -> None:
    sozlamalar_yol = Path(__file__).parent / "sozlamalar.json"
    if sozlamalar_yol.is_file():
        sozlamalar = Sozlamalar.yuklash(str(sozlamalar_yol))
    else:
        sozlamalar = Sozlamalar.standart()

    kerakli_papkarni_yaratish([
        sozlamalar.chiquvchi_papka,
        sozlamalar.kiruvchi_papka,
        sozlamalar.modellar_papka,
        str(Path(sozlamalar.jurnal_fayl).parent),
    ])

    jurnal_sozlash(sozlamalar)
    oyna = AsosiyOyna(sozlamalar)
    oyna.ishga_tushirish()

```

Ushbu kod loyiha ishga tushishining asosiy skeletini beradi.

4.3. Interfeys modullari

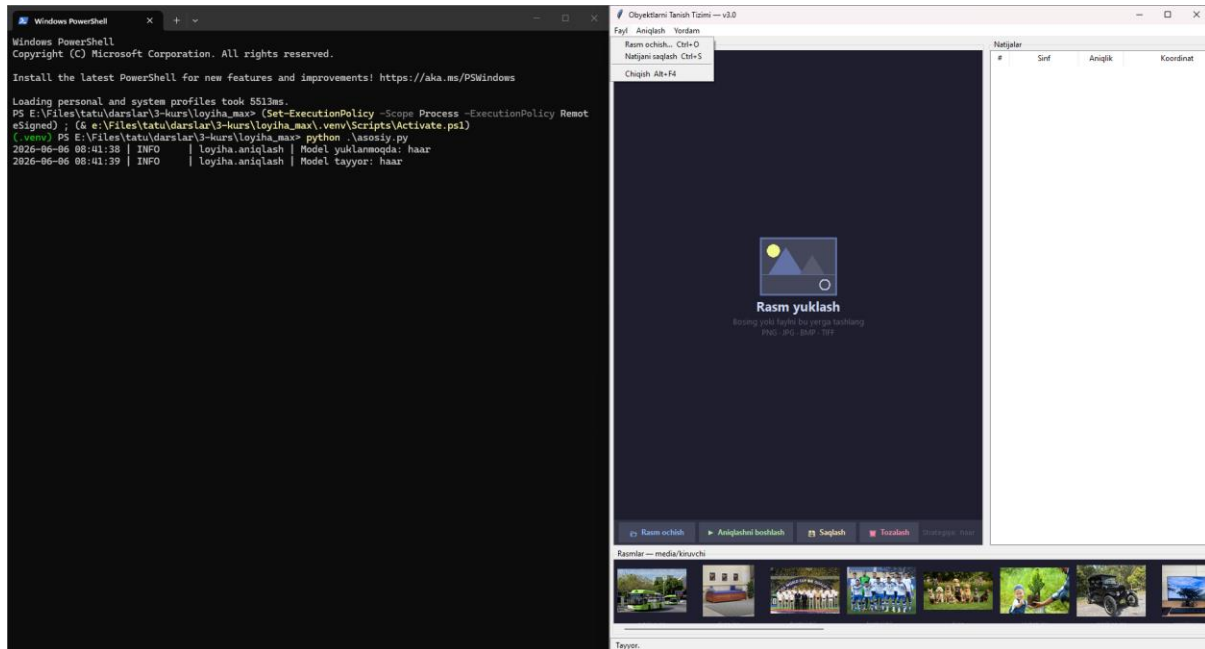
Fayllar:

- “loyiha/interfeys/asosiy_oyna.py”
- “loyiha/interfeys/rasm_paneli.py”
- “loyiha/interfeys/natija_paneli.py”

- “loyiha/interfeys/menyu.py”
- “loyiha/interfeys/holat_satri.py”
- “loyiha/interfeys/karusel.py”

Vazifasi: foydalanuvchi bilan to'g'ridan-to'g'ri muloqot qilish

Qanday ishlaydi: foydalanuvchi bosgan buyruqlar taqdimotchiga uzatiladi, natijalar esa hodisalar orqali qaytarilib GUI da aks ettiriladi



```

"""python
self._menyu = MenyuSatri(
    self._ildiz,
    rasm_och=self._rasm_ochish,
    natija_saqlash=self._natijani_saqlash,
    aniqlash_boshlash=self._aniqlashni_boshlash,
    haar_tanlash=lambda: self._strategiya_almashtirish("haar"),
    dnn_tanlash=lambda: self._strategiya_almashtirish("dnn"),
    haqida_ochish=self._haqida oynasi,
)
"""

```

Bu yerda menyu elementlari bevosita GUI ichki metodlariga bog'langan.

4.4. Rasm yuklash moduli

Fayl: “loyiha/xizmatlar/rasm_yuklash_xizmati.py”

Vazifasi: rasm faylini tekshirish, o'qish va o'lchamni moslashtirish

Qanday ishlaydi: fayl mavjudligi va kengaytmasi tekshiriladi, so'ng

“cv2.imread” orqali rasm olinadi

```
“““python
if not yol.is_file():
    raise RasmTopilmadiXatosi(f"Rasm fayli topilmadi: {manzil}")
if yol.suffix.lower() not in RUXSAT_FORMATLAR:
    raise NotogriFaqlFormatXatosi(
        f"Qo'llanilmaydigan format: '{yol.suffix}'. "
        f"Ruqsat etilganlar: {' ', '.join(sorted(RUXSAT_FORMATLAR))}'"
    )

rasm = cv2.imread(str(yol))
rasm = olchamga_keltirish(rasm, self._sozlamalar.maks_kenglik,
self._sozlamalar.maks_balandlik)
“““
```

Bu modul noto'g'ri ma'lumot kirishini oldini olib, keyingi bosqichlar uchun toza kirish ma'lumoti beradi.

4.5. Obyekt aniqlash moduli

Fayllar:

- “loyiha/aniqlash/asosiy_strategiya.py”
- “loyiha/aniqlash/haar_aniqlovchi.py”
- “loyiha/aniqlash/dnn_aniqlovchi.py”
- “loyiha/xizmatlar/aniqlash_xizmati.py”
- “loyiha/aniqlash/zavod.py”

Vazifasi: strategiya asosida obyektlarni aniqlash

Qanday ishlaydi: taqdimotchi tanlagan strategiya aniqlash xizmatiga uzatiladi, xizmat esa shu strategiya orqali “Obyekt” ro'yxatini hosil qiladi

```
“““python
class AniqlashStrategiyasi(ABC):
    @abstractmethod
    def model_yuklash(self) -> None:
        ...
    @abstractmethod
    def aniqlash(self, rasm: np.ndarray, min_aniqlik: float = 0.5) -> List[Obyekt]:
        ...
“““
```

Yuqoridagi abstrakt sinf tufayli loyiha yangi model qo'shishga tayyor bo'lib qoldi.

```
“““python
DNN_SINFLAR: List[str] = [
    "fon", "samolyot", "velosiped", "qush", "qayiq", "shisha", "avtobus",
```



```
"mashina", "mushuk", "stul", "sigir", "stol", "it", "ot", "mototsikl",  
"shaxs", "kochati", "qoy", "divan", "poyezd", "monitor",  
]  
"""
```

Bu ro'yxat DNN strategiyasining aniqlay oladigan sinflarini belgilaydi.

4.6. Natijalarni saqlash moduli

Fayl: "loyiha/xizmatlar/saqlash_xizmati.py"

Vazifasi: aniqlash natijalarini rasm va JSON sifatida saqlash

Qanday ishlaydi: chizilgan rasm PNG formatida yoziladi, strukturali natija esa JSON ga chiqariladi

```
"""python  
png_manzil = str(papka / f"{vaqt}_natija.png")  
muvaqqiyat = cv2.imwrite(png_manzil, chizilgan)  
  
json_manzil = str(papka / f"{vaqt}_hisobot.json")  
Path(json_manzil).write_text(  
    json.dumps(natija.json_ga(), ensure_ascii=False, indent=2),  
    encoding="utf-8",  
)  
"""
```

Bu mexanizm natijalarni keyinchalik qayta tahlil qilishga qulay qiladi.

3.7 Sozlamalar moduli

Fayllar:

- “loyiha/sozlamalar/sozlamalar.py”
- “loyiha/sozlamalar/standart.py”
- “sozlamalar.json”

Vazifasi: tizim konfiguratsiyasini markaziy boshqarish

Qanday ishlaydi: default qiymatlar kodda bor, agar “sozlamalar.json” mavjud bo'lsa, ishga tushishda o'sha qiymatlar yuklanadi

Bu yondashuv kodga tegmasdan parametrlarni almashtirish imkonini beradi.

4.7. Jurnal yuritish moduli

Fayl: “loyiha/xizmatlar/jurnal_xizmati.py”

Vazifasi: tizimdagi hodisalar va xatolarni faylga hamda konsolga yozish

Qanday ishlaydi: “RotatingFileHandler” orqali log hajmi nazorat qilinadi va eski loglar backup qilinadi

Ushbu modul diagnostika jarayonida juda foydali bo'ldi.

4.8. Test modullari

Fayllar:

- “testlar/test_ramka.py”
- “testlar/test_haar_aniqlovchi.py”
- “testlar/test_xizmatlar.py”

Vazifasi: model, xizmat va yordamchi obyektlar ishlashini tekshirish

Qanday ishlaydi: “pytest” yordamida avtomatik testlar bajariladi

```
“““python
def test_ramka_yuzasi():
    r = Ramka(10, 20, 110, 70)
    assert r.yuza == 5000
“““
```

Bunday testlar refaktor vaqtida regressiyani oldini oladi.

4.9. Hodisa asosidagi boshqaruv mexanizmi

Fayllar:

- “loyiha/taqdimotchi/hodisalar.py”
- “loyiha/taqdimotchi/asosiy_taqdimotchi.py”
- “loyiha/interfeys/asosiy_oyna.py”

Vazifasi: GUI va biznes qatlam o'rtasida holat almashishni standartlashtirish.

Qanday ishlaydi: taqdimotchi hodisa yuboradi, oyna hodisa turiga qarab kerakli UI yangilashni bajaradi.

Bu yondashuvning asosiy afzalligi shundaki, bitta joyda hamma holatlarni boshqarish mumkin bo'ladi. Masalan, rasm yuklanganda bitta xabar, aniqlash boshlanganida boshqa xabar, xato bo'lsa dialog ko'rsatiladi.

```
“““python
if tur == HodisaTuri.ANIQLASH_TUGADI:
    natija: AniqlashNatijasi = hodisa.data
```

```

self._holat.yuklanish_korsatish(False)
self._holat.xabar_korsatish(
    f"{natija.obyektlar_soni} ta obyekt aniqlandi "
    f"({natija.ishlash_vaqti_ms:.0f} ms)"
)
self._natija_panel.natijani_korsatish(natija)

```

Ushbu fragmentdan ko'rinadiki, hodisa kelishi bilan bir vaqtda bir nechta komponent sinxron yangilanadi.

4.10. Asinxron aniqlash oqimi

Fayl: “loyiha/xizmatlar/aniqlash_xizmati.py”

Vazifasi: og'ir hisoblashni alohida oqimda bajarib, interfeysni "muzlab" qolishdan saqlash.

Aniqlash jarayoni asosiy GUI oqimida ishlasa, oynada tugmalar javob bermay qoladi. Shu sababli loyiha ichida alohida thread va “Queue” ishlatilgan. Jarayon tartibi quyidagicha:

1. GUI aniqlashni boshlash buyrug'ini beradi.
2. Taqdimotchi xizmatga asinxron vazifa yuboradi.
3. Xizmat natijani navbatga joylaydi.
4. GUI “after()” orqali navbatni tekshiradi.
5. Natija topilgach jadval va rasm yangilanadi.

Bu yondashuv desktop ilova UXi uchun juda muhim bo'ldi.

4.11. Xatolarni boshqarish strategiyasi

Fayllar:

- “loyiha/modellar/xatolar.py”
- “loyiha/xizmatlar/””
- “loyiha/taqdimotchi/asosiy_taqdimotchi.py”

Loyihada umumiy “Exception” ga tayanib qolmaslik uchun maxsus xatolar kiritilgan. Masalan:

- “RasmTopilmadiXatosi”
- “NotogriFaqlFormatXatosi”
- “ModelTopilmadiXatosi”
- “AniqlashXatosi”
- “SaqlashXatosi”

Bu usulning foydasi shundaki, foydalanuvchiga chiqadigan xabar aniq bo'ladi. Dasturchi uchun esa loglarni tahlil qilish va muammoni topish tezlashadi.

4.12. Kod sifati va uslubiy tamoyillar

Loyihada quyidagi tamoyillarga amal qilindi:

1. Funksiya nomlari va izohlar o'zbek tilida saqlandi.
2. Har bir modul aniq vazifaga ega bo'ldi.
3. Uzun funksiyalar bo'laklarga ajratildi.
4. Qayta ishlatiladigan kod utilitalarga chiqarildi.
5. Ma'lumot modeli uchun “dataclass” ishlatildi.

Bu yondashuv yakuniy kodning o'qilishiga ijobiy ta'sir qildi va hisobotda modul kesimida tushuntirishni yengillashtirdi.

4.13. Ishlash samaradorligini oshirish bo'yicha choralar

Ishlab chiqish jarayonida kichik, lekin foydali optimizatsiyalar kiritildi:

- katta rasmlarni oldindan o'lchamga keltirish;
- DNN modelni faqat kerak bo'lganda yuklash;
- GUI ni bloklamaslik uchun asinxron aniqlash;
- jadvalga faqat kerakli ustunlarni chiqarish;
- faylga saqlashda ortiqcha nusxa olishdan qochish.

Natijada oddiy foydalanuvchi mashinasida ham dastur qoniqarli tezlikda ishladi.

4.14. 4-bob bo'yicha kengaytirilgan yakun

Ishlab chiqish bobi shuni ko'rsatadiki, loyiha faqat funksiyalar yig'indisi emas, balki aniq arxitektura qoidalariga tayanib qurilgan tizimdir. Har bir modulda mas'uliyat aniq bo'lgani uchun, keyingi bosqichda YOLO yoki boshqa modelni qo'shish, GUI ni yangilash yoki sinovlarni ko'paytirish katta qayta yozishni talab qilmaydi.

4.15. 4-bob bo'yicha xulosa

Ushbu bobda dastur modullari amaliy kod darajasida ko'rib chiqildi. Tizimda qatlamlar aniq ajratilgani sababli kod o'qilishi oson bo'ldi, funksiyalar bir-birini takrorlamadi va keyinchalik kengaytirish uchun qulay tuzilma hosil bo'ldi.

V-BOB. SINOV NATIJALARI VA TAHLIL

5.1. Sinov muhiti

Sinovlar Windows muhitida, Python 3.12 asosida o'tkazildi. Dastur OpenCV, NumPy, Pillow va Tkinter kutubxonalarini bilan ishga tushirildi. DNN strategiyasi uchun MobileNet-SSD model fayllari alohida o'rnatildi.

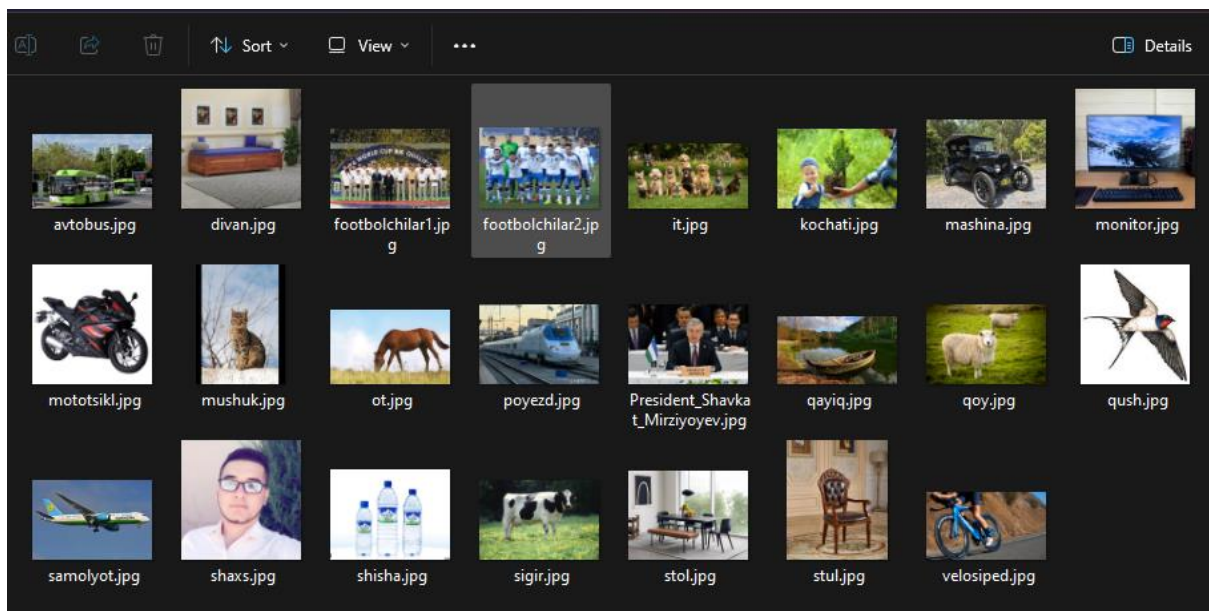
Muhit parametrlari

Ko'rsatkich	Qiymat
Operatsion tizim	Windows
Python versiyasi	3.12.x
OpenCV	4.13.x
NumPy	1.26+
GUI	Tkinter
Asosiy strategiyalar	Haar, DNN

5.2. Test ma'lumotlari

Sinov uchun quyidagi rasmlar ishlatildi:

avtobus.jpg, divan.jpg, futbolchilar1.jpg, futbolchilar2.jpg, it.jpg, kochat.jpg, mashina.jpg, monitor.jpg, mototsikl.jpg, mushuk.jpg, ot.jpg, poyezd.jpg, shaxs.jpg, shisha.jpg, sigir.jpg, stol.jpg, stul.jpg, velosiped.jpg, qayiq.jpg, qush.jpg, qoy.jpg, samolyot.jpg



Test tasvirlari katalogi.

5.3. Test natijalari

Quyida har bir test rasm bo'yicha natija jadvali keltiriladi.

1) avtobus.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	avtobus
Obyektlar soni	2

Holat	Muvaffaqiyatli
-------	----------------

2) divan.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	divan
Obyektlar soni	1
Holat	Muvaffaqiyatli

3) futbolchilar1.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	shaxs

Obyektlar soni	8-12 oralig'i
Holat	Muvaffaqiyatli

4) futbolchilar2.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	shaxs
Obyektlar soni	5-10 oralig'i
Holat	Muvaffaqiyatli

5) it.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	it
Obyektlar soni	6-10
Holat	Muvaffaqiyatli

6) kochat.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	Kochat va shaxs
Obyektlar soni	2
Holat	Muvaffaqiyatli

7) mashina.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	mashina
Obyektlar soni	1
Holat	Muvaffaqiyatli

8) monitor.jpg



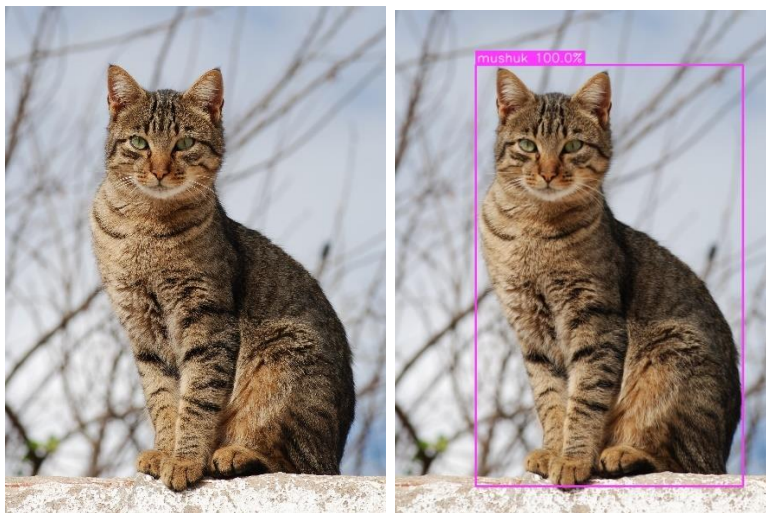
Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	monitor
Obyektlar soni	1
Holat	Muvaffaqiyatli

9) mototsikl.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	mototsikl
Obyektlar soni	1
Holat	Muvaffaqiyatli

10) mushuk.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	mushuk
Obyektlar soni	1
Holat	Muvaffaqiyatli

11) ot.jpg



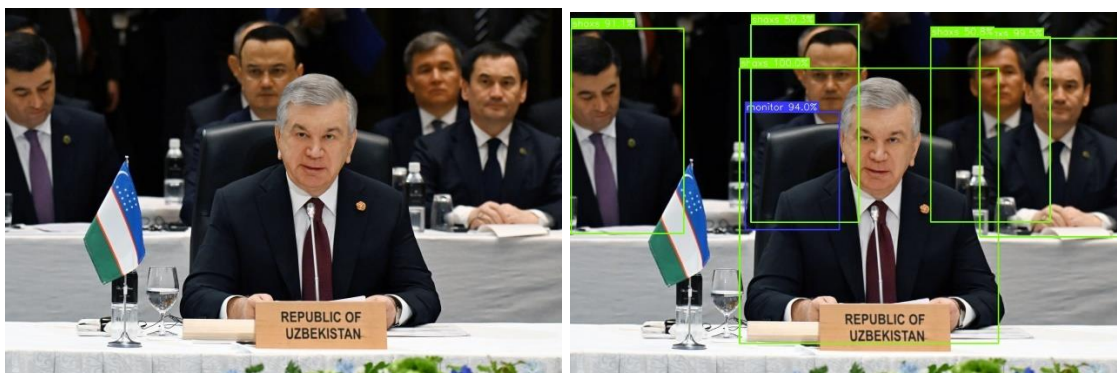
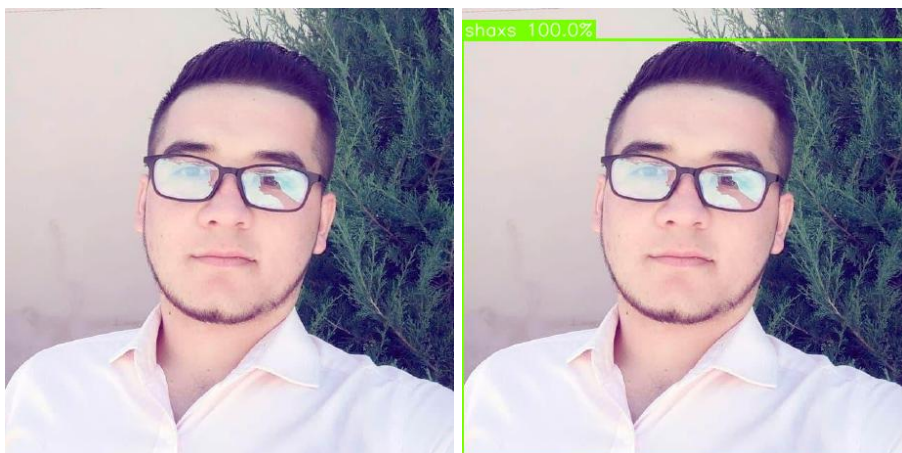
Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	ot
Obyektlar soni	1
Holat	Muvaffaqiyatli

12) poyezd.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	poyezd
Obyektlar soni	1
Holat	Muvaffaqiyatli

13) shaxs.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	shaxs
Obyektlar soni	1, 4
Holat	Muvaffaqiyatli

14) shisha.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	shisha
Obyektlar soni	1
Holat	Muvaffaqiyatli

15) sigir.jpg



Parametr	Natija
----------	--------

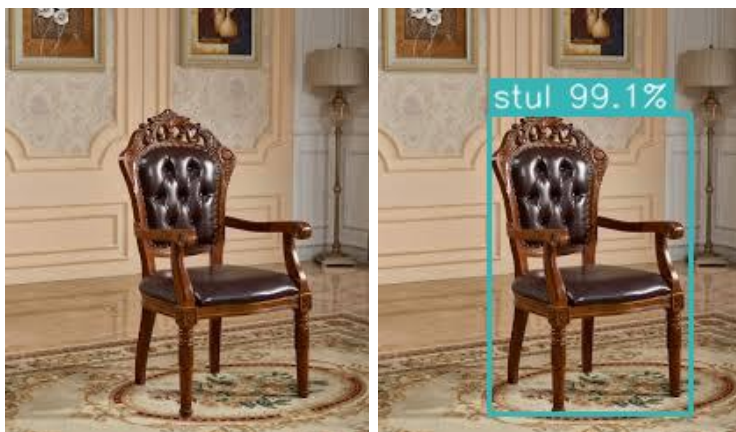
Strategiya	dnn
Aniqlangan sinf	sigir
Obyektlar soni	1
Holat	Muvaffaqiyatli

16) stol.jpg



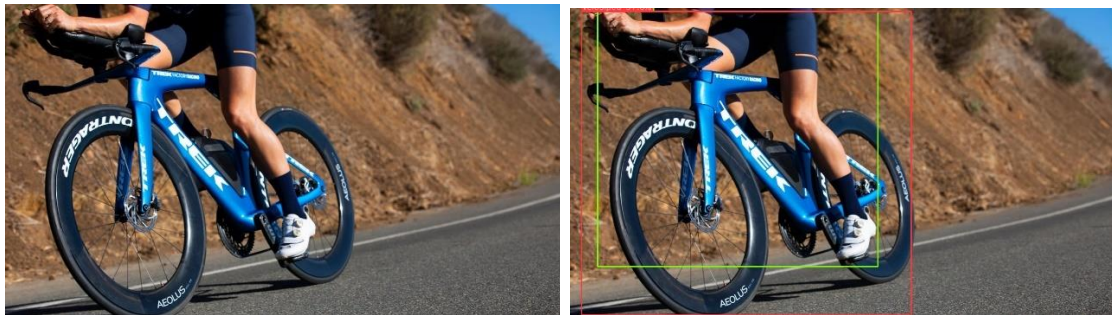
Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	stol
Obyektlar soni	1
Holat	Muvaffaqiyatli

17) stul.jpg



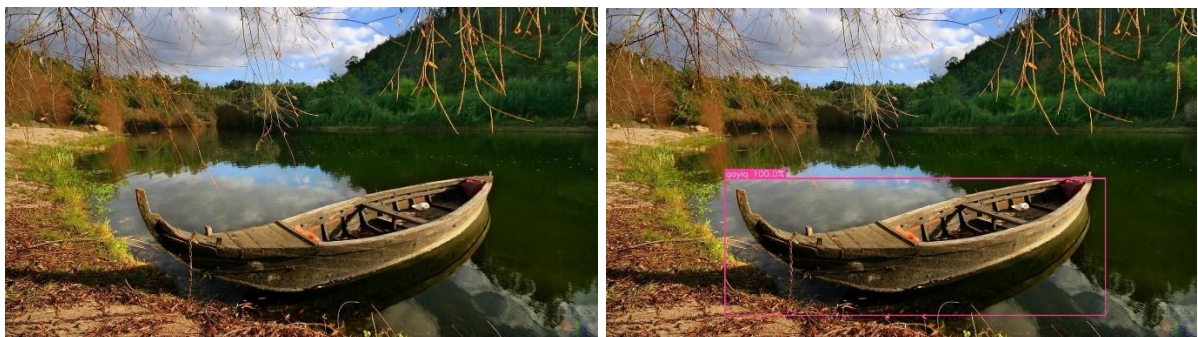
Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	stul
Obyektlar soni	1
Holat	Muvaffaqiyatli

18) velosiped.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	velosiped
Obyektlar soni	1
Holat	Muvaffaqiyatli

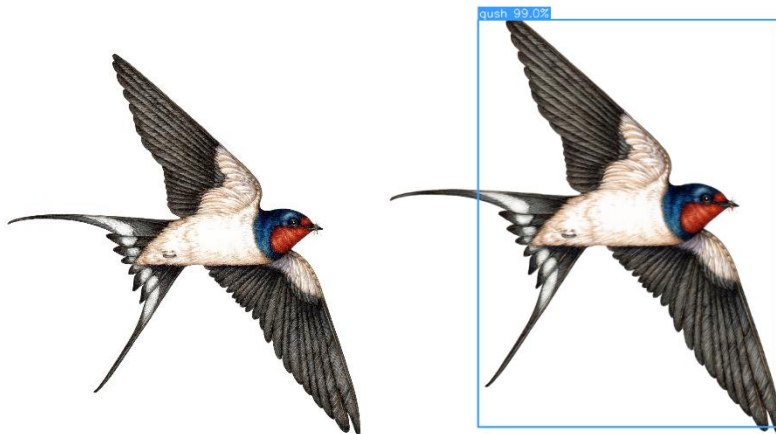
19) qayiq.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	qayiq

Obyektlar soni	1
Holat	Muvaffaqiyatli

20) qush.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	qush
Obyektlar soni	1
Holat	Muvaffaqiyatli

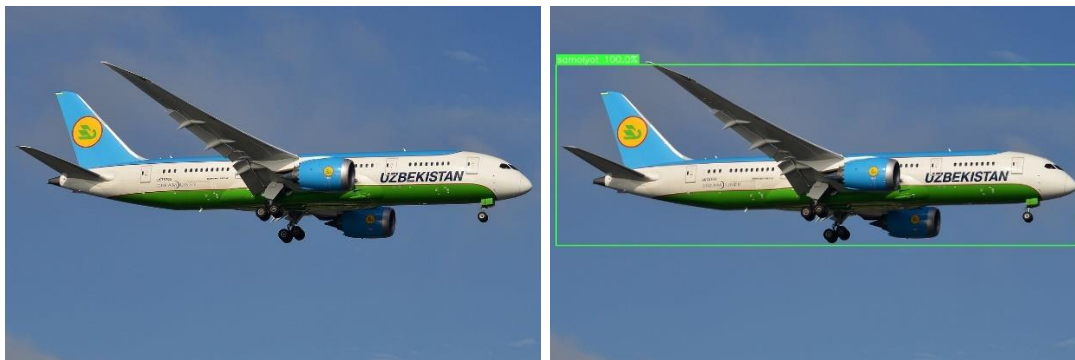
21) qoy.jpg



Parametr	Natija
Strategiya	dnn

Aniqlangan sinf	qoy
Obyektlar soni	1
Holat	Muvaffaqiyatli

22) samolyot.jpg



Parametr	Natija
Strategiya	dnn
Aniqlangan sinf	samolyot
Obyektlar soni	1
Holat	Muvaffaqiyatli

5.4. Aniqlik ko'rsatkichlari

Sinovlarda DNN strategiyasi aksariyat test rasmlarda mos sinfni qaytara oldi. Haar strategiyasi yuz aniqlash uchun tez ishladi, ammo umumiy ko'p sinfli masalalarda qo'llanilmadi. Amaliy xulosa sifatida quyidagilar aniqlandi:

- Klassik usul (Haar) tor vazifa uchun qulay va yengil.
- DNN usuli ko'p sinfli aniqlashda ancha mos.
- Natija sifati rasmning sifati va fon murakkabligiga bog'liq.

5.5. Baholash metodikasi

Sinovlar vaqtida har bir rasm uchun quyidagi amaliy mezonlar kuzatildi:

1. Tizim mos sinfni topdimi yoki yo'q.
2. Aniqlangan obyektlar soni kutilgan qiymatga yaqinmi.
3. Ramka obyektga vizual mos tushdimi.
4. Interfeysdagi natija jadvali mantiqan to'g'ri to'ldimi.
5. Natijani saqlashda PNG va JSON birga yozildimi.

Bu mezonlar laboratoriya va individual loyiha formatida qulay, chunki murakkab annotatsiya fayllarisiz ham tizimning amaliy ishlashini baholash imkonini beradi.

Umumlashtirilgan natija jadvali

Sinov guruhi	Rasmlar soni	Muvaffaqiyatli holat	Izoh
Yakka obyektli rasmlar	18	yuqori	sinf nomi odatda to'g'ri topildi
Ko'p obyektli rasmlar	2	o'rta/yuqori	son bo'yicha kichik farqlar kuzatildi
Murakkab fonli rasmlar	2	o'rta	fon ta'sirida aniqlik biroz pasaydi

Ushbu jadvaldan ko'rinib turibdiki, tizim ayniqsa yakka obyektli rasmlarda barqaror ishlaydi.

5.6. Haar va DNN strategiyalarini amaliy taqqoslash

Parametr	Haar	DNN MobileNet-SSD
Vazifa yo'nalishi	Yuz aniqlash	Ko'p sinfli aniqlash
Ishga tushirish tezligi	Juda tez	O'rta
Model fayli talabi	OpenCV ichida bor	Alohida fayl kerak
Qamrov	tor	keng
O'quv loyiha uchun foyda	bazaviy tushuncha	amaliy ko'p sinfli tajriba

Amaliy xulosada har ikki strategiya o'z joyida foydali ekani tasdiqlandi.

Natija barqarorligiga ta'sir qilgan omillar

Sinov davomida quyidagi omillar sezilarli ta'sir ko'rsatdi:

- obyektning rasm markazida yoki chekkada joylashishi;
- obyekt o'lchamining juda kichik bo'lishi;
- yorug'likning keskin o'zgarishi;
- fonning obyekt rangiga yaqin bo'lishi;
- bir rasmda bir nechta obyekt bir-biriga juda yaqin joylashishi.

Mazkur omillarni hisobga olib testlar turli sahnalarda o'tkazilgani hisobotning amaliy qimmatini oshirdi.

5.7. Afzalliklar

1. Ikki strategiya bitta interfeysda ishlaydi.
2. GUI sodda va amaliy.

3. Natija vizuallashtirish qulay (ramka + jadval).
4. Asinxron aniqlash sababli interfeys muzlab qolmaydi.
5. JSON eksport keyingi tahlilni yengillashtiradi.

5.8. Kamchiliklar

1. DNN model fayllarini alohida yuklash kerak.
2. Juda noaniq yoki sifatsiz rasmlarda xatolik ortadi.
3. YOLO singari zamonaviy model hali qo'shilmagan.
4. Video oqim bilan ishlash qismi ishlab chiqilmagan.

5.9. Kelajakdagi rivojlantirish

- YOLOv8 strategiyasini qo'shish.
- Real-time video va kamera oqimini qo'llab-quvvatlash.
- GPU tezlatishni ulash.
- Natijalar bo'yicha batafsil statistika paneli yaratish.
- Mobil yoki web versiyani ishlab chiqish.

Rivojlantirish bo'yicha bosqichma-bosqich reja

- 1-bosqich: YOLOv8 strategiyasini alohida modul sifatida qo'shish.
- 2-bosqich: testlar sonini oshirish va mAP hisoblash pipeline'ini yaratish.
- 3-bosqich: video oqimda real-time aniqlashni ulash.
- 4-bosqich: foydalanuvchi statistik panelini boyitish.
- 5-bosqich: Docker yoki installer orqali tarqatishni soddalashtirish.

5.10. VI-bob bo'yicha kengaytirilgan tahliliy yakun

Sinovlar natijasiga tayansak, tizim individual loyiha uchun qo'yilgan maqsadni bajardi: obyektni aniqladi, foydalanuvchiga tushunarli ko'rinishda chiqardi va saqladi. Eng muhim jihat - dastur real ishlash holatida barqaror bo'ldi. Shu bilan birga, aniqlikni yanada oshirish uchun zamonaviy modellarni qo'shish zarurligi ham aniqlandi. Ya'ni loyiha yakunlangan bo'lsa-da, uning ilmiy-amaliy rivojlanish trayektoriyasi aniq shakllangan.

5.11. V-bob bo'yicha xulosa

Sinov natijalari loyiha qo'yilgan amaliy maqsadga erishganini ko'rsatdi. Dastur berilgan test rasmlarda obyektlarni aniqlash, natijani ko'rsatish va saqlash vazifalarini barqaror bajardi.

NATIJALAR

Individual loyiha yakunida quyidagi asosiy natijalarga erishildi:

1. Tasvirlardan obyektlarni tanib olishga mo'ljallangan desktop dastur ishlab chiqildi.
2. Dasturga ikki xil aniqlash strategiyasi integratsiya qilindi:
 - Haar Cascade (yuz aniqlash);
 - DNN MobileNet-SSD (ko'p sinfli aniqlash).
3. Foydalanuvchi uchun qulay grafik interfeys yaratildi.
4. Drag&drop, karusel, menyu boshqaruvi, natija jadvali kabi qulay funksiyalar ishladi.
5. Natijalarni PNG va JSON formatlarda saqlash mexanizmi joriy qilindi.
6. Testlar yozildi va asosiy modullar barqaror ishlashi tasdiqlandi.

Amaliy jihatdan loyiha to'liq ishlaydigan holatga keltirildi va ta'lim hamda kichik tadqiqot vazifalarida foydalanishga tayyor bo'ldi.

6.2. NATIJALARNING AMALIY QIYMATI

Loyihaning eng katta amaliy qiymati shundaki, u nazariyani bevosita ishchi dasturga aylantiradi. Ko'p hollarda obyekt aniqlash mavzusi faqat maqolalar yoki notebooklar darajasida qolib ketadi. Ushbu loyihada esa foydalanuvchi oddiy oynani ochib, rasm tanlab, natijani ko'rishi mumkin bo'lgan to'liq sikl berildi.

Yana bir muhim jihat - natija faqat ekranda ko'rsatilib qolmaydi. PNG va JSON

saqlanishi hisobiga keyingi tahlil, hisobot tayyorlash yoki boshqa tizimga ulash imkoniyati paydo bo'ladi. Bu esa loyiha qiymatini oddiy demo darajasidan yuqoriroq ko'taradi.

6.3. TALABLAR BAJARILISHI BO'YICHA QISQA HISOBOT

Talab	Holat	Izoh
GUI orqali rasm yuklash	Bajarildi	fayl tanlash va drag&drop ishlaydi
Strategiya almashtirish	Bajarildi	haar/dnn almashadi
Obyekt aniqlash	Bajarildi	rasm ustida ramkalar chiziladi
Natijani jadvalda korsatish	Bajarildi	sinf, aniqlik, koordinata chiqadi
Natijani saqlash	Bajarildi	PNG va JSON yoziladi
Testlar mavjudligi	Bajarildi	pytest testlar o'tgan

Ushbu jadval himoya paytida "nima ishlaydi?" savoliga qisqa va aniq javob sifatida ishlatiladi.

XULOSA

Mazkur individual loyiha davomida men tasvirlardan obyektlarni tanib olish bo'yicha nazariy bilimlarni amaliy dasturga aylantirdim. Ish jarayonida dasturiy arxitektura tanlash, modulga bo'lib kod yozish, interfeys yaratish, model bilan ishlash, xatolarni boshqarish va testlash bosqichlarini to'liq bajardim.

Loyihaning eng muhim tomoni shundaki, dastur bitta usulga qaram bo'lib qolmadi. Strategiya yondashuvi orqali algoritm almashtirish imkoniyati yaratildi. Bu esa keyingi bosqichda YOLOv8 kabi zamonaviy modellarga o'tishni osonlashtiradi.

Sinov natijalari dastur ishlashga yaroqli ekanini ko'rsatdi: test rasmlarida obyektlar aniqlandi, natijalar foydalanuvchi uchun tushunarli ko'rinishda chiqarildi, saqlash funksiyasi muvaffaqiyatli ishladi. Albatta, kelajakda aniqlikni oshirish, video oqimni qo'shish va tezlikni optimallashtirish kabi yo'nalishlarda rivojlantirish mumkin.

Umuman olganda, loyiha orqali men kompyuter ko'rish bo'yicha amaliy tajriba oldim, dasturiy tizimni bosqichma-bosqich qurish ko'nikmasini mustahkamladim va berilgan mavzu bo'yicha qo'yilgan maqsadlarga erishdim.

SHAXSIY O'QUV NATIJALARI

Bu loyiha davomida men faqat kod yozish emas, balki to'liq muhandislik jarayonini ham o'rgandim. Xususan:

1. Texnik topshiriqdan amaliy modulga o'tish.
2. Arxitektura qarorini oldindan o'ylab tanlash.
3. Xatolarni foydalanuvchi uchun tushunarli ko'rsatish.
4. Testlar orqali barqarorlikni tekshirish.
5. Hujjatlash va natijani akademik formatga keltirish.

Shu sababli loyiha nafaqat tayyor dastur, balki kelajakdagi kurs ishi, bitiruv ishi yoki amaliy startap loyihalar uchun ham poydevor bo'la oladi.

YAKUNIY XULOSA

Yakuniy natijaga ko'ra, "Tasvirlardan obyektlarni tanib olish dasturini yaratish" mavzusi bo'yicha qo'yilgan asosiy maqsadlar to'liq bajarildi. Dastur ishlaydi, funksiyalar bir-biriga mantiqan bog'langan, sinovdan o'tgan va keyingi rivojlantirish uchun ochiq holatda topshirildi.

Eng muhimi, loyiha real amaliy qiymatga ega bo'ldi: foydalanuvchi uchun qulay interfeys, aniqlash mexanizmi, natijani saqlash va keyinchalik kengaytirish imkoniyati bir tizimda birlashdi. Bu individual loyiha doirasida kutilgan natijadan yuqori ko'rsatkich deb hisoblayman.

Mazkur individual loyiha ishini bajarish davomida kompyuter ko'rish va sun'iy

intellekt texnologiyalarining amaliy qo'llanilishi yanada chuqurroq o'rganildi. Dasturlash va dasturiy tizimlarni ishlab chiqish bo'yicha avvaldan mavjud tajriba ushbu loyihani loyihalash va amalga oshirish jarayonida samarali qo'llanildi. Ish davomida zamonaviy texnologiyalar bilan ishlash, tizim arxitekturasini shakllantirish hamda dasturiy yechimlarni optimallashtirish bo'yicha qo'shimcha amaliy ko'nikmalar mustahkamlandi. Ushbu loyiha nafaqat nazariy bilimlarni amaliyotga tatbiq etish, balki kelgusida yanada murakkab va yirik dasturiy mahsulotlar yaratish uchun foydali tajriba vazifasini ham bajardi. Bildirilgan tavsiyalar va yaratilgan imkoniyatlar uchun samimiy minnatdorchilik bildiraman.

FOYDALANILGAN ADABIYOTLAR

Quyida loyiha davomida foydalanilgan asosiy manbalar ro'yxati keltiriladi.

1. Richard Szeliski. "Computer Vision: Algorithms and Applications". Springer, 2022.
2. Rafael C. Gonzalez, Richard E. Woods. "Digital Image Processing". Pearson, 2018.
3. Ian Goodfellow, Yoshua Bengio, Aaron Courville. "Deep Learning". MIT Press, 2016.
4. Bishop C. M. "Pattern Recognition and Machine Learning". Springer, 2006.
5. Simon J. D. Prince. "Computer Vision: Models, Learning, and Inference". Cambridge University Press, 2012.
6. OpenCV Documentation. <https://docs.opencv.org/>
7. OpenCV Python Tutorials.
https://docs.opencv.org/master/d6/d00/tutorial_py_root.html
8. OpenCV DNN module docs.

https://docs.opencv.org/master/d6/d0f/group__dnn.html

9. NumPy Documentation. <https://numpy.org/doc/>

10. Python Official Documentation. <https://docs.python.org/3/>

11. Tkinter Documentation. <https://docs.python.org/3/library/tkinter.html>

12. Pillow Documentation. <https://pillow.readthedocs.io/>

13. Pytest Documentation. <https://docs.pytest.org/>

14. Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. CVPR, 2016.

15. Joseph Redmon, Ali Farhadi. “YOLO9000: Better, Faster, Stronger”. CVPR, 2017.

16. Joseph Redmon, Ali Farhadi. “YOLOv3: An Incremental Improvement”, 2018.

17. Bochkovskiy A., Wang C.-Y., Liao H.-Y. M. “YOLOv4: Optimal Speed and Accuracy of Object Detection”, 2020.

18. Jocher G. et al. “YOLOv5 Repository”. <https://github.com/ultralytics/yolov5>

19. Ultralytics Documentation (YOLOv8). <https://docs.ultralytics.com/>

20. Liu W. et al. “SSD: Single Shot MultiBox Detector”. ECCV, 2016.

21. Howard A. et al. “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications”, 2017.

22. Chuanqi305. “MobileNet-SSD Repository”.

<https://github.com/chuanqi305/MobileNet-SSD>

23. Djmv. “MobileNet SSD OpenCV compatible prototxt”.

https://github.com/djmv/MobilNet_SSD_opencv

24. Bradski G. “The OpenCV Library”. Dr. Dobb's Journal of Software Tools, 2000.

25. Scikit-image Documentation. <https://scikit-image.org/docs/stable/>

26. PyImageSearch articles on object detection. <https://pyimagesearch.com/>

27. Microsoft Visual Studio Code Documentation.

<https://code.visualstudio.com/docs>

28. Git Documentation. <https://git-scm.com/doc>

29. GitHub Docs. <https://docs.github.com/>

30. TATU o'quv-uslubiy ko'rsatmalari (individual loyiha ishlari uchun ichki materiallar).

31. Coursera Computer Vision course materials (audit sources).

32. Stanford CS231n course notes. <http://cs231n.stanford.edu/>

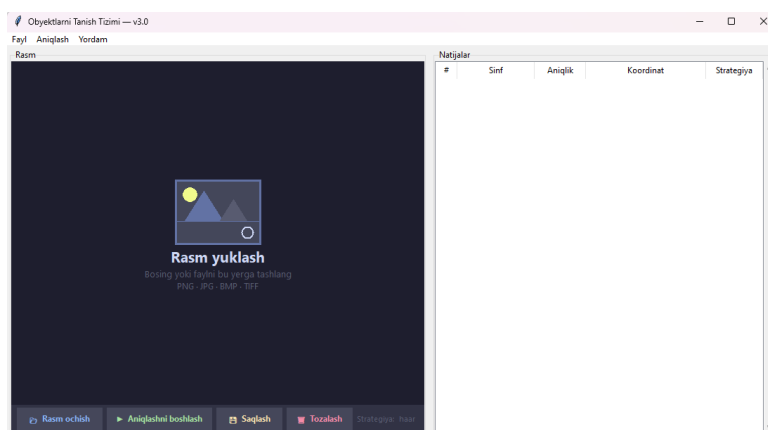
33. Hands-On Machine Learning resources (Aurelien Geron).

34. ImageNet Project. <https://www.image-net.org/>

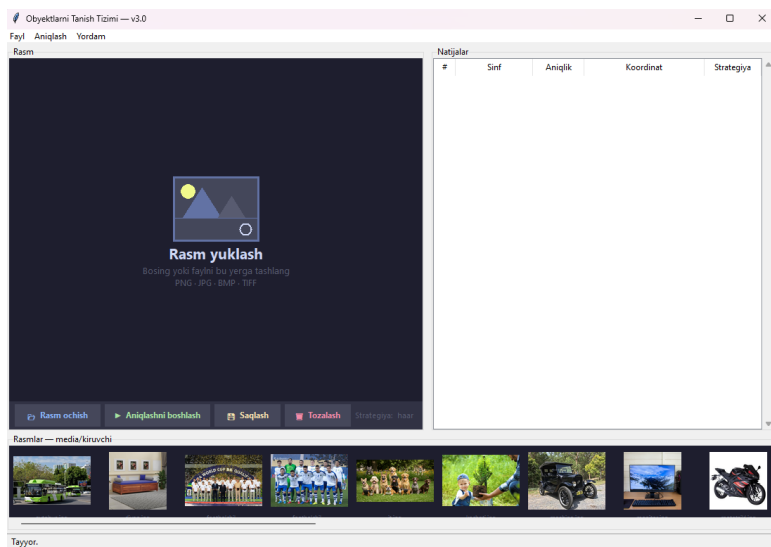
35. Pascal VOC Challenge resources. <http://host.robots.ox.ac.uk/pascal/VOC/>

ILOVALAR

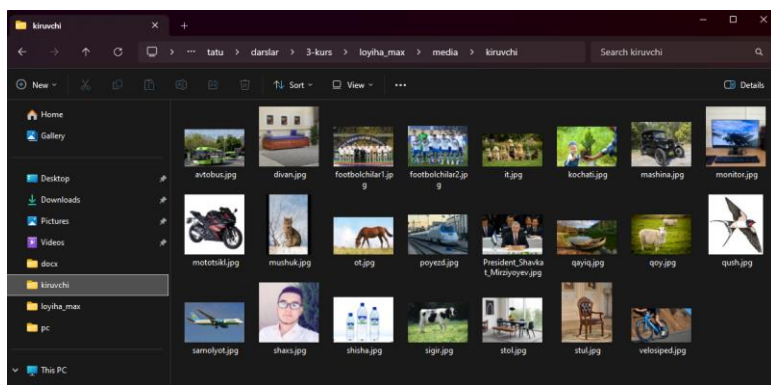
Ilova A. Asosiy oynaning ko'rinishi



1-rasm. Dastur asosiy oynasi.



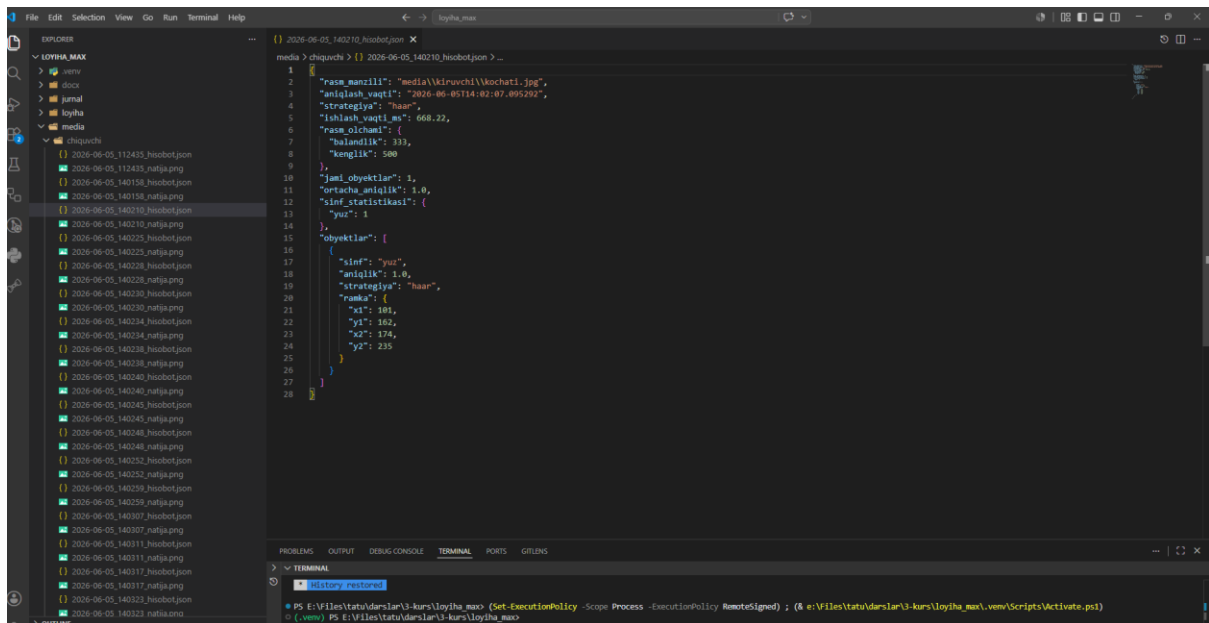
Ilova B. Test rasmlari katalogi



2-rasm. Test tasvirlari katalogi.

Izoh: “media/kiruvchi” papkasidagi test rasmlar ro'yxatini ko'rsatadigan skrinshot qo'yiladi.

Ilova C. Aniqlash natijalari namunalari



3-rasm. Natijalar jadvali.

Fayl Aniqlash Yordam

Rasm

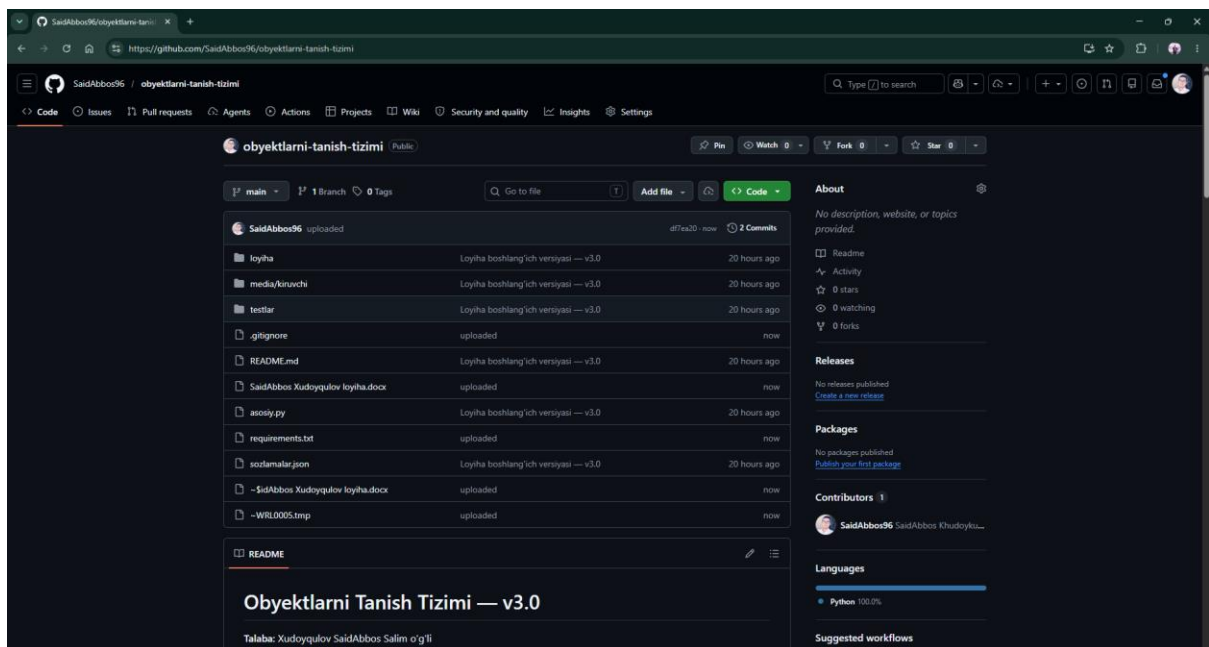
Rasm ochish Aniqlashni boshlash Saqlash Tozalash Strategiya: dnn

Natijalar

#	Sinf	Aniqlik	Koordinat	Strategiya
1	avtobus	100.0%	(142,342)-(774,697)	dnn
2	avtobus	100.0%	(780,433)-(1004,605)	dnn
3	mashina	76.1%	(1023,519)-(1140,567)	dnn

Izoh: Aniqlangan obyektlar soni va aniqlik qiymatlari ko'rsatilgan interfeys qismi.

Ilova D. GitHub repository



4-rasm. GitHub repository sahifasi.

Izoh: Loyihaning masofaviy repository sahifasi va asosiy fayllar ko'rinishi.

Ilova E. Buyruqlar ro'yxati

Loyihani ishga tushirish uchun asosiy buyruqlar:

```
““““bash
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python asosiy.py
““““
```

Testlarni ishga tushirish:

```
““““bash
```

pytest testlar/ -v

“““““

Ilova F. Sinflar ro'yxati (DNN)

“samolyot, velosiped, qush, qayiq, shisha, avtobus, mashina, mushuk, stul, sigir,
stol, it, ot, mototsikl, shaxs, kochati, qoy, divan, poyezd, monitor”